

Assignment 1: Sequential Baselines and Benchmarking

ITCS 4145 — Parallel Programming — Fall 2026

Instructor: S M Ferdous · Released: Sep 3, 2026 · Due: Sept 18, 2026

Total: 15 points (5 parts $\times$ 3 points)

1 Scope

Implement five sequential algorithms (also referred as kernels) in C or C++, measure their running times on the Galaxy cluster (Centaurus partition), and report the results. These implementations are the baselines against which the parallel versions in later assignments will be compared.

This assignment asks you to *implement and measure*. Where a part asks you to compare variants, report what you observe; a full explanation of the hardware behaviour behind the measurements is not required, and we will return to these results later in the semester once the relevant material has been covered.

Standard-library routines may be used as comparison points where a part specifies one, but not in place of the required implementations.

2 Logistics

2.1 Execution environment

All reported measurements must be taken on the cluster.

```
ssh <niner-id>@hpc-student.uncc.edu
```

Benchmarks must be run through `sbatch` or `salloc`, not on the head node. Compiling on the head node is acceptable.

2.2 Submission

A single archive named `a1_<niner-id>.tgz` or `.zip` on Canvas:

```
a1_<niner-id>/
report.pdf    the graded document
Makefile      builds every binary from clean, with no edits
README.md     how to build and how to run
src/          sources
data/         raw timing output in CSV form
scripts/      sbatch scripts, plotting scripts
```

2.3 Grading

Score	Criterion
3	Correct, with only minor issues
2	Mostly correct, with many issues
1	Mostly incorrect or incomplete
0	No submission, or no significant submission

3 Measurement requirements

These apply to every part.

1. **Clock.** Use `clock_gettime(CLOCK_MONOTONIC, ...)`. `clock()` (measures CPU time) and `time()` (does not have enough resolution) are not acceptable. A working timer is provided in the starter kit.
2. **Timed region.** Allocation, input generation, input restoration, verification, and printing must lie outside the timed region.
3. **Warm-up.** Run the kernel once without timing it before measuring.
4. **Repetitions.** At least ten timed repetitions. Report the average, median, the minimum, and the maximum.
5. **Use the result.** Print the kernel's result or add it to a running total. If the result is never used, the compiler may remove the computation and the measured time will be near zero.
6. **Restore the input.** Kernels that modify their input (prefix sum, sorting) must have the input restored before each repetition, outside the timed region.
7. **Compile flags.** State the compiler version and the flags used. Unless a part says otherwise, use `-O2` or `-O3`.

3.1 Reporting rates

Report a rate in addition to elapsed time, using these definitions.

Kernel	Rate
Array maximum	ns/t bytes/s (the <i>memory bandwidth</i> ; s is the element size)
Prefix sum	n/t elements/s
Matrix multiplication	$2MKN/t$ floating-point operations/s
Merge sort	n/t elements/s
BFS	(edges inspected)/ t , called TEPS

4 Part 1 — Array maximum

[3 pts]

Find the largest element of an array of n elements.

1. Implement the kernel for `int` arrays and for `double` arrays.
2. Implement two versions of the inner loop:

```
/* A */          /* B */
if (a[i] > m) m = a[i];    m = (a[i] > m) ? a[i] : m;
```

3. Measure every combination at $n = 10^6$, 10^7 , and 10^8 . Report elapsed time and rate.
4. Repeat with three inputs: sorted, reverse-sorted, and random. Report whether the input order changes the running time.
5. Report whether the `int` and `double` versions differ in rate, and by how much.
6. Plot rate (see section 3.1) against n .

4.1 Memory bandwidth and the effect of array size

The maximum kernel does almost no arithmetic: it reads each element once and compares it. Its speed is therefore set by how fast the machine can deliver data from memory. The rate

$$\text{bandwidth} = \frac{n \times \text{sizeof}(\text{element})}{t} \quad \text{bytes per second}$$

measures exactly that, and because this kernel only reads and never writes, the byte count is unambiguous.

A processor does not fetch data from main memory one element at a time. It keeps recently used data in small, fast memories called caches, of which there are usually three levels, each larger and slower than the one before. An array that fits in a cache is delivered much faster than one that does not.

7. **Sweep the array size.** Measure the bandwidth of the `double` branchless version for arrays from 2 KB up to at least 256 MB, doubling the size at each step. Plot bandwidth against array size in bytes, with a logarithmic horizontal axis.
8. Run `1scpu` and record the L1, L2, and L3 cache sizes of the node. Mark them on your plot, and state how the sizes at which the bandwidth drops compare with them.

5 Part 2 — Exclusive prefix sum**[3 pts]**

Given $a[0..n-1]$, replace each element by the sum of all elements up to it: $a[i] \leftarrow \sum_{k < i} a[k]$. Here $a[0] = 0$ and $a[n] = \sum_{0 \leq k < n} a[k]$

1. Implement an inplace prefix sum algorithm. An inplace algorithm do not create additional output array, but compute the quantity in the input array.
2. Measure, for `int` and `double`, at $n = 10^6$, 10^7 , and 10^8 .
3. Compile and measure at three optimization levels: `-O0`, `-O2`, and `-O3`. Report all three.

6 Part 3 — Dense matrix multiplication

[3 pts]

Compute $C = AB$, where A is $M \times K$, B is $K \times N$, and C is $M \times N$. **Store all three matrices row-major in single flat allocations.**

6.1 Notation

The product is defined by

$$C[i][j] = \sum_{k=0}^{K-1} A[i][k] \cdot B[k][j],$$

so the three loop indices have fixed roles:

Index	Range	Role
i	$0 \leq i < M$	row of A , row of C
j	$0 \leq j < N$	column of B , column of C
k	$0 \leq k < K$	column of A , row of B ; summed over, absent from C

Each triple (i, j, k) contributes one product to one output element, so all six orderings of the three loops compute the same result.

6.2 Requirements

1. Implement all six loop orders: `ijk`, `ikj`, `jik`, `jki`, `kij`, `kji`.
2. Verify that all six produce the same result.
3. Measure all six at (M, K, N) equal to $(256, 256, 256)$, $(512, 512, 512)$, and $(1024, 1024, 1024)$, and at one shape where M , K , and N are not all equal. You can use random non-zero integer for elements.
4. Report the rate for each, and plot it.
5. Some of the six orders perform very similarly to each other. Group the orders by their measured rate, state which loop index the orders in each group have in common, and give the ratio between the fastest and slowest group.

7 Part 4 — Merge sort

[3 pts]

Sort an array of n integers.

1. Implement a recursive merge sort. Write the merge step as its own function.
2. Implement two versions: one that allocates its temporary array inside every recursive call, and one that allocates a single temporary array once and reuses it. Measure both and report the difference.
3. Measure at $n = 10^6$, 10^7 , and 10^8 , and compare against `qsort` (C) or `std::sort` (C++).
4. Measure with four inputs: random, already sorted, reverse-sorted, and all elements equal. Report the differences.
5. Verify both that the output is sorted and that it contains the same elements as the input. Checking only that the output is sorted is not sufficient, since an array of all zeros is sorted.

8 Part 5 — Breadth-first search

[3 pts]

Compute the BFS distance from a source vertex to every reachable vertex of an undirected graph.

8.1 Graph representation

Use compressed sparse row (CSR). For a graph with n vertices and m edges. Those who will implement in C++, you may use `std::vector`.

```
typedef struct {
    int n;           /* vertices */
    long m;          /* edges */
    long *offsets;   /* length n+1 */
    int *adj;        /* the neighbours of v are stored in
                     /* adj[offsets[v] .. offsets[v+1]-1] */
} graph_t;
```

Do not use an $n \times n$ adjacency matrix: at $n = 10^6$ it would need 10^{12} entries.

8.2 Requirements

1. Generate two graphs with 2^{20} vertices and average degree 16, using Erdos-Renyi and RMAT models and store it in matrix market files (.mtx format). You may use networkx (networkit) in python for generating graph and writing it as an .mtx file.
2. Write a graph reader from .mtx file, that will read the graph and store it in CSR format.
3. Implement BFS level by level, using an array to hold the current frontier. Report the number of levels, the size of each level's frontier, and the fraction of vertices reached.
4. Report TEPS/s, using the number of edges your traversal actually inspected as the numerator.
5. Run BFS from at least 16 randomly chosen starting vertices of nonzero degree, and report the minimum, median, and maximum TEPS across them.
6. Compare the TEPS of the two generated graphs and report which is faster and by how much.

9 Report

1. **Setup**, stated once: the node used, compiler version, compile flags, number of repetitions, and which statistic you report.
2. **One section per part**: what you implemented, how you checked it was correct, and the measurements as tables or figures.
3. **A short closing section** listing the rate achieved by each of the five kernels, so they can be compared side by side.

Figures must have labelled axes with units and a caption stating what the figure shows. Use a logarithmic scale on any axis spanning more than one decade.